



BPI Challenge 2019¹

Mina Nayernia

Business Information Systems
Project Report²
June 2023
University of Milan

¹ BPI link: https://data.4tu.nl/articles/_/12715853/1

² Colab link: [BIS2019.ipynb](#)

Challenge Description	2
Data	2
The Challenges	3
Phase 1 - Answering challenge 1	4
Dataset exploration and preprocessing	4
1.Time frame	4
2.Compliant	4
3.Completeness	9
Variant analysis	12
Clusters	13
Process Models(BPMNs)	13
Cluster 1.1	14
Cluster 1.2.1	14
Cluster 1.2.2	15
Cluster 1.2.3	15
Cluster 2.1.1	15
Cluster 2.1.2	15
Cluster 2.2.1	15
Cluster 2.2.2	16
Cluster 2.2.3	16
Cluster 3	16
Cluster 4	16
Validation of case group and process model quality	17
Phase 2 - Answering challenge 2	18
Throughput Time Analysis	18
Phase 3 - Answering challenge 3	18
Deviations	18
Conclusion	19

Challenge Description

Data

For the BPI Challenge 2019, There is a collected data from a large multinational company operating from The Netherlands in the area of coatings and paints.

In the data, each purchase order (or purchase document) contains one or more line items. For each line item, there are roughly four types of flows in the data:

1. **3-way matching, invoice after goods receipt.**
For these items, the value of the goods receipt message should be matched against the value of an invoice receipt message and the value put during creation of the item (indicated by both the GR-based flag and the Goods Receipt flags set to true).
2. **3-way matching, invoice before goods receipt.**
Purchase Items that do require a goods receipt message, while they do not require GR-based invoicing (indicated by the GR-based IV flag set to false and the Goods Receipt flags set to true). For such purchase items, invoices can be entered before the goods are receipt, but they are blocked until goods are received. This unblocking can be done by a user, or by a batch process at regular intervals. Invoices should only be cleared if goods are received and the value matches with the invoice and the value at creation of the item.
3. **2-way matching (no goods receipt needed)**
For these items, the value of the invoice should match the value at creation (in full or partially until PO value is consumed), but there is no separate goods receipt message required (indicated by both the GR-based flag and the Goods Receipt flags set to false).
4. **Consignment**
For these items, there are no invoices on PO level as this is handled fully in a separate process. Here we see GR indicator is set to true but the GR IV flag is set to false and also we know by item type (consignment) that we do not expect an invoice against this item.

Unfortunately, the complexity of the data goes further than just this division in four categories. For each purchase item, there can be many goods receipt messages and corresponding invoices which are subsequently paid. Consider for example the process of paying rent. There is a Purchase Document with one item for paying rent, but a total of 12 goods receipt messages with (cleared) invoices with a value equal to 1/12 of the total amount. For logistical services, there may even be hundreds of goods receipt messages for one line item.

Overall, for each line item, the amounts of the line item, the goods receipt messages (if applicable) and the invoices have to match for the process to be compliant.

For each purchase item (or case) the following attributes are recorded:

- **concept:name:** A combination of the anonymized purchase document id and the anonymized item id,
- **Purchasing Document:** The anonymized purchasing document ID,
- **Item:** The anonymized item ID,
- **Item Type:** The type of the item,
- **GR-Based Inv. Verif.:** Flag indicating if GR-based invoicing is required (see above),
- **Goods Receipt:** Flag indicating if 3-way matching is required (see above),
- **Source:** The anonymized source system of this item,
- **Doc. Category name:** The name of the category of the purchasing document,
- **Company:** The anonymized subsidiary of the company from where the purchase originated,
- **Spend classification text:** A text explaining the class of purchase item,
- **Spend area text:** A text explaining the area for the purchase item,
- **Sub spend area text:** Another text explaining the area for the purchase item,
- **Vendor:** The anonymized vendor to which the purchase document was sent,
- **Name:** The anonymized name of the vendor,
- **Document Type:** The document type,
- **Item Category:** The category as explained above (3-way with GR-based invoicing, 3-way without, 2-way, consignment).

The Challenges

The company is interested in answers to three main questions:

1. Is there a collection of process models which together properly describe the process in this data. Based on the four categories above, at least 4 models are needed, but any collection of models that together explain the process well is appreciated. Preferably, the decision which model explains which purchase item best is based on properties of the item.
2. What is the throughput of the invoicing process, i.e. the time between goods receipt, invoice receipt and payment (clear invoice)? To answer this, a technique is sought to

match these events within a line item, i.e. if there are multiple goods receipt messages and multiple invoices within a line item, how are they related and which belong together?

3. Finally, which Purchase Documents stand out from the log? Where are deviations from the processes discovered in (1) and how severe are these deviations? Deviations may be according to the prescribed high-level process flow, but also with respect to the values of the invoices. Which customers produce a lot of rework as invoices are wrong, etc.?

Phase 1 - Answering challenge 1

Dataset exploration and preprocessing

Not all of the presented events could be used for the analyses, because they may be noncompliant to the basic rules mentioned in the description of challenge or even not complete due to user mistakes of entering data.

1. Events did not occur in the extraction timeframe
2. Cases are not complete
3. Cases are not compliant

1. Time frame

First I checked the timing (Following rules) of events for the entire DataFrame and then I did the categorizing.

- Final date 2019-01-28 23:59:00+00:00 (because the data was submitted in that time)
- beginning of 2018-01-01 00:00:00+00:00

```
#checking the time frame
filterd_log_df_by_time = log_df[log_df['time:timestamp'] > '2018-01-01 00:00:00+00:00'] #checking the start time
filterd_log_df_by_time = filterd_log_df_by_time[filterd_log_df_by_time['time:timestamp'] < '2019-01-28 23:59:00+00:00'] #checking the end time
#cathegorising
log_3way_invoice_after_good_receipt = filterd_log_df_by_time[filterd_log_df_by_time['case:Item Category'] == '3-way match, invoice after GR']
log_3way_invoice_before_good_receipt = filterd_log_df_by_time[filterd_log_df_by_time['case:Item Category'] == '3-way match, invoice before GR']
log_2way = filterd_log_df_by_time[filterd_log_df_by_time['case:Item Category'] == '2-way match']
log_Consignment = filterd_log_df_by_time[filterd_log_df_by_time['case:Item Category'] == 'Consignment']
```

Reduction of 327 rows by timing filter.

2. Compliant

Below I have mentioned the compliant cases (Or rules to identify them) and noncompliant cases are filtered by applying these rules:

All, except 2-way match:

- Number of GRs > 0

I checked if it has at least one “Record Goods Receipt” in each case.

```
#compliance check
#All, except 2-way match: Number of GRs > 0

mask = log_3way_invoice_after_good_receipt["concept:name"].isin(["Record Goods Receipt"])
test = log_3way_invoice_after_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered_log_3way_invoice_after_good_receipt = log_3way_invoice_after_good_receipt[log_3way_invoice_after_good_receipt["case:concept:name"].isin(test)]
# print(len(filtered_log_3way_invoice_after_good_receipt))
# print(len(log_3way_invoice_after_good_receipt))
#result: filtered 319140 rows into 317171

mask = log_3way_invoice_before_good_receipt["concept:name"].isin(["Record Goods Receipt"])
test = log_3way_invoice_before_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered_log_3way_invoice_before_good_receipt = log_3way_invoice_before_good_receipt[log_3way_invoice_before_good_receipt["case:concept:name"].isin(test)]
# print(len(filtered_log_3way_invoice_before_good_receipt))
# print(len(log_3way_invoice_before_good_receipt))
#result: filtered 1234494 rows into 1198905

mask = log_Consignment["concept:name"].isin(["Record Goods Receipt"])
test = log_Consignment[mask]["case:concept:name"].unique() #those cases that are compliant
filtered_log_Consignment = log_Consignment[log_Consignment["case:concept:name"].isin(test)]
# print(len(filtered_log_Consignment))
# print(len(log_Consignment))
#result: filtered 36083 rows into 34009
```

Reduction of 1969 in 3way_after , 35589 in 3way_before, 2074 in consignment by this filter.

All, except Consignment:

- Number of Clear Invoice events > 0
- Number of invoice receipts > 0

Checking if there is at least one “Clear Invoice” and one “Record Invoice Receipt” in each case.

```
mask = filtered_log_3way_invoice_after_good_receipt["concept:name"].isin(["Clear Invoice"])
test = filtered_log_3way_invoice_after_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered2_log_3way_invoice_after_good_receipt = filtered_log_3way_invoice_after_good_receipt[filtered_log_3way_invoice_after_good_receipt["case:concept:name"].isin(test)]
# print(len(filtered2_log_3way_invoice_after_good_receipt))
# print(len(filtered_log_3way_invoice_after_good_receipt))
#result: filtered 317171 rows into 155839

mask = filtered2_log_3way_invoice_after_good_receipt["concept:name"].isin(["Record Invoice Receipt"])
test = filtered2_log_3way_invoice_after_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered3_log_3way_invoice_after_good_receipt = filtered2_log_3way_invoice_after_good_receipt[filtered2_log_3way_invoice_after_good_receipt["case:concept:name"].isin(test)]
# print(len(filtered3_log_3way_invoice_after_good_receipt))
# print(len(filtered2_log_3way_invoice_after_good_receipt))
#result: filtered 155839 rows into 155818
```

Reduction of 161353 events in 3way_after by this filter.

```

mask = filtered_log_3way_invoice_before_good_receipt["concept:name"].isin(["Clear Invoice"])
test = filtered_log_3way_invoice_before_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered2_log_3way_invoice_before_good_receipt = filtered_log_3way_invoice_before_good_receipt[filtered_log_3way_invoice_before_good_receipt["case:concept:name"].isin(test)]
# print(len(filtered2_log_3way_invoice_before_good_receipt))
# print(len(filtered_log_3way_invoice_before_good_receipt))
#result: filtered 1198905 rows into 1039611

mask = filtered2_log_3way_invoice_before_good_receipt["concept:name"].isin(["Record Invoice Receipt"])
test = filtered2_log_3way_invoice_before_good_receipt[mask]["case:concept:name"].unique() #those cases that are compliant
filtered3_log_3way_invoice_before_good_receipt = filtered2_log_3way_invoice_before_good_receipt[filtered2_log_3way_invoice_before_good_receipt["case:concept:name"].isin(test)]

# print(len(filtered3_log_3way_invoice_before_good_receipt))
# print(len(filtered2_log_3way_invoice_before_good_receipt))
#result: filtered 1039611 rows into 1039563

```

Reduction of 159342 events in way_before by this filter.

```

mask = log_2way["concept:name"].isin(["Clear Invoice"])
test = log_2way[mask]["case:concept:name"].unique() #those cases that are compliant
filtered_log_2way = log_2way[log_2way["case:concept:name"].isin(test)]
# print(len(filtered_log_2way))
# print(len(log_2way))
#result: filtered 5879 rows into 2492

mask = filtered_log_2way["concept:name"].isin(["Record Invoice Receipt"])
test = filtered_log_2way[mask]["case:concept:name"].unique() #those cases that are compliant
filtered2_log_2way = filtered_log_2way[filtered_log_2way["case:concept:name"].isin(test)]
# print(len(filtered2_log_2way))
# print(len(filtered_log_2way))
#result: filtered 2492 rows into 2492 (no changes)

```

No changes.

Consignment:

- Cumulated value of the GRs divided by the number of GRs = PO item value

```

#compliance check
#Consignment: Cumulated value of the GRs divided by the number of GRs = PO item value

# filtered_log_Consignment.columns
filtered_log_Consignment["Cumulative net worth (EUR)"].unique()

#for all of lines : the Cumulated value == 0 => so no need to check it ????????

array([0.])

```

No changes.

3-way match, invoice after GR, 3-way match, invoice before GR:

- PO item value = Cumulated value of the invoice receipts divided by the number of invoice receipts

First finding the incompliant:

```

current_po = ""
current_po_value = 0
current_cumulated_value_of_invoice_receipts = 0
num_invoice_receipts = 0
incompliants = []
# print("before1" , len(filtered3_log_3way_invoice_before_good_receipt))

for index, row in filtered3_log_3way_invoice_before_good_receipt.iterrows():
    # print(row["concept:name"])
    if row["concept:name"] == "Create Purchase Order Item": #found new po
        #first check the rule for previous PO
        if current_po != "":
            if current_po_value != current_cumulated_value_of_invoice_receipts / num_invoice_receipts : #incompliant
                incompliants.append(current_po)

            current_po = row["case:concept:name"] #save the current po id to check then for invoice reciepts
            current_po_value = row["Cumulative net worth (EUR)"]
            current_cumulated_value_of_invoice_receipts = 0
            num_invoice_receipts = 0

        elif row["concept:name"] == "Record Invoice Receipt" and row["case:concept:name"] == current_po:
            num_invoice_receipts += 1
            current_cumulated_value_of_invoice_receipts += int(row["Cumulative net worth (EUR)"])

```

Then filtering them:

```

# The incompliants is empty => nothing to filter
#deleting incompliants
incompliants = set(list(dict.fromkeys(incompliants)))
all = set(filtered3_log_3way_invoice_before_good_receipt["case:concept:name"])
compliant = list(all - incompliants)
mask = filtered3_log_3way_invoice_before_good_receipt["case:concept:name"].isin(compliant)
filtered3_log_3way_invoice_before_good_receipt = filtered3_log_3way_invoice_before_good_receipt[mask]
# print("after1" ,len(filtered3_log_3way_invoice_before_good_receipt) )

```

No change in 3way_before.

(The code is the same for 3way_after, but the result is : Reduction of 1016 events)

- Cumulated value of the GRs = Cumulated value of the invoice receipts

Finding incompliants:


```

# # 2.Cumulated value of the GRs = Cumulated value of the invoice receipts
current_po = ""
current_cumulated_GRs = 0
current_cumulated_invoice = 0
incompliers = []
# print("before2" , len(filtered3_log_3way_invoice_before_good_receipt))

for index, row in filtered3_log_3way_invoice_before_good_receipt.iterrows():

    if row["concept:name"] == "Create Purchase Order Item": #found new po
        if current_po != "":

            if current_cumulated_GRs != current_cumulated_invoice :
                # print("current_cumulated_GRs", current_cumulated_GRs)
                # print("current_cumulated_invoice" , current_cumulated_invoice)
                incompliers.append(current_po)

            current_po = row["case:concept:name"]
            current_cumulated_GRs = 0
            current_cumulated_invoice = 0

        elif row["concept:name"] == "Record Goods Receipt" and row["case:concept:name"] == current_po:
            current_cumulated_GRs += int(row["Cumulative net worth (EUR)"])

        elif row["concept:name"] == "Record Invoice Receipt" and row["case:concept:name"] == current_po:
            current_cumulated_invoice += int(row["Cumulative net worth (EUR)"])

```

Then filtering:

```

#deleting incompliers
incompliers = set(list(dict.fromkeys(incompliers)))
all = set(filtered3_log_3way_invoice_before_good_receipt["case:concept:name"])
compliers = list(all - incompliers)

mask = filtered3_log_3way_invoice_before_good_receipt["case:concept:name"].isin(compliers)
filtered3_log_3way_invoice_before_good_receipt = filtered3_log_3way_invoice_before_good_receipt[mask]

# print("after2" , len(filtered3_log_3way_invoice_before_good_receipt))
#result: reduced by 4578

```

Reduced by 4578 events in 3way_before.

(The code is the same for 3way_after, but the result is : Reduction of 5410)

- Number of GRs = Number of invoice receipts

Finding incompliers:

```

# 3.Number of GRs = Number of invoice receipts
current_po = ""
number_of_GRs = 0
number_if_invoice_receipt = 0
incompliants = []
# print("before",len(filtered3_log_3way_invoice_before_good_receipt))

for index, row in filtered3_log_3way_invoice_before_good_receipt.iterrows():

    if row["concept:name"] == "Create Purchase Order Item": #found new po
        if current_po != "":

            if number_of_GRs != number_if_invoice_receipt :
                incompliants.append(current_po)

            current_po = row["case:concept:name"]
            number_of_GRs = 0
            number_if_invoice_receipt = 0

        elif row["concept:name"] == "Record Goods Receipt" and row["case:concept:name"] == current_po:
            number_of_GRs += 1

        elif row["concept:name"] == "Record Invoice Receipt" and row["case:concept:name"] == current_po:
            number_if_invoice_receipt += 1

```

Filtering:

```

incompliants = set(list(dict.fromkeys(incompliants)))
all = set(filtered3_log_3way_invoice_before_good_receipt["case:concept:name"])
compliants = list(all - incompliants)

mask = filtered3_log_3way_invoice_before_good_receipt["case:concept:name"].isin(compliants)
filtered3_log_3way_invoice_before_good_receipt = filtered3_log_3way_invoice_before_good_receipt[mask]

# print("after",len(filtered3_log_3way_invoice_before_good_receipt))
#result: from 1023911 to 958608

```

Reduction of 65303 in 3way_before events.

(The code is the same for 3way_after, but the result is :No changes)

3.Completeness

1. Incomplete PO items can be recognized by their end activities, using function **filter_end_activities** , which is assumed to be Clear invoice for all cases except Consignment PO items, which do not involve an invoicing process.

```

#2-way
# end_activities = pm4py.get_end_activities(log_2way)
# print(end_activities)
# print("log_2way" , len(log_2way))
log_2way = pm4py.filter_end_activities(log_2way , ["Clear Invoice"])
# print("log_2way" , len(log_2way))
#result: from 5879 to 1016

#3-way, before
# print(len(log_3way_invoice_before_good_receipt))
log_3way_invoice_before_good_receipt = pm4py.filter_end_activities(log_3way_invoice_before_good_receipt , ["Clear Invoice"])
# print(len(log_3way_invoice_before_good_receipt))
#result: from 1234494 to 1027034

#3-way, after
# print(len(log_3way_invoice_after_good_receipt))
log_3way_invoice_after_good_receipt = pm4py.filter_end_activities(log_3way_invoice_after_good_receipt , ["Clear Invoice"])
# print(len(log_3way_invoice_after_good_receipt))
#result: from 319140 to 136016

```

In general we had a reduction of 395447 cases by this filter.
(4863 in log_2way , 207460 in log_3way_before, 183124 in log_3way_after)

2. The recording of a GR is assumed to be the end activity for Consignment PO items, using function **filter_end_activities**.

```

# print(len(log_Consignment))
log_Consignment = pm4py.filter_end_activities(log_Consignment , ["Record Goods Receipt"])
# print(len(log_Consignment))
#result: from 34009 to 32591

```

Reduction of 1418 cases in log_consignment by this filter

3. More than one GR and invoice receipt are possible in the purchasing process, which may also lead to multiple clearings of invoices.
 - In 3-way match: invoice after goods receipt order items, invoices have to follow goods receipts, so incomplete orders are those, in which fewer invoice receipts were received than goods receipt, reduced by cancellations.

So first, I found the incomplete POs:

```
current_po = ""
number_of_GRs = 0
number_of_invoices = 0
incompletes = []
for index, row in log_3way_invoice_after_good_receipt.iterrows():
    # print(row["concept:name"])
    if row["concept:name"] == "Create Purchase Order Item": #found new po
        #first check the rule for previous PO
        if current_po != "":
            if number_of_GRs != number_of_invoices :
                incompletes.append(current_po)

        current_po = row["case:concept:name"]
        number_of_GRs = 0
        number_of_invoices = 0

    elif row["concept:name"] == "Record Invoice Receipt" and row["case:concept:name"] == current_po:
        number_of_invoices +=1

    elif row["concept:name"] == "Record Goods Receipt" and row["case:concept:name"] == current_po:
        number_of_GRs +=1

    elif row["concept:name"] == "Cancel Invoice Receipt" and row["case:concept:name"] == current_po:
        number_of_invoices -=1

    elif row["concept:name"] == "Cancel Goods Receipt" and row["case:concept:name"] == current_po:
        number_of_GRs -=1
```

Then filter them(remove) from the DataFrame:

```
# print(len(log_3way_invoice_after_good_receipt))
incompletes = set(list(dict.fromkeys(incompletes)))
all = set(log_3way_invoice_after_good_receipt["case:concept:name"])
completes = list(all - incompletes)
mask = log_3way_invoice_after_good_receipt["case:concept:name"].isin(completes)
log_3way_invoice_after_good_receipt = log_3way_invoice_after_good_receipt[mask]
# print(len(log_3way_invoice_after_good_receipt))

#result: from 136016 to 56156
```

Reduction of 79860 events by this filter.

- For 3-way match: invoice before goods receipt order items, the order of GRs and invoice receipts is insignificant, so both a lower count of goods receipts or invoice receipts compared to the respective other implies an incomplete PO item.

First finding the incomplete POs:

```
current_po = ""
number_of_GRs = 0
number_of_invoices = 0
incompletes = []
for index, row in log_3way_invoice_before_good_receipt.iterrows():
    # print(row["concept:name"])
    if row["concept:name"] == "Create Purchase Order Item": #found new po
        #first check the rule for previous PO
        if current_po != "":
            if number_of_GRs != number_of_invoices :
                incompletes.append(current_po)

        current_po = row["case:concept:name"]
        number_of_GRs = 0
        number_of_invoices = 0

    elif row["concept:name"] == "Record Invoice Receipt" and row["case:concept:name"] == current_po:
        number_of_invoices +=1

    elif row["concept:name"] == "Record Goods Receipt" and row["case:concept:name"] == current_po:
        number_of_GRs +=1

    elif row["concept:name"] == "Cancel Invoice Receipt" and row["case:concept:name"] == current_po:
        number_of_invoices -=1

    elif row["concept:name"] == "Cancel Goods Receipt" and row["case:concept:name"] == current_po:
        number_of_GRs -=1
```

Then filtering them:

```
# print(len(log_3way_invoice_before_good_receipt))
incompletes = set(list(dict.fromkeys(incompletes)))
all = set(log_3way_invoice_before_good_receipt["case:concept:name"])
completes = list(all - incompletes)
mask = log_3way_invoice_before_good_receipt["case:concept:name"].isin(completes)
log_3way_invoice_before_good_receipt = log_3way_invoice_before_good_receipt[mask]
# print(len(log_3way_invoice_before_good_receipt))

#result: from 1027034 to 987422
```

Reduction of 39612 events by this filter.

Variant analysis

At first I was trying to explore the variants by functions `get_variants` and `abstract_variants` in `pm4py`. There were 490 variants just for `3way_after` (that doesn't make sense to use this method for getting variants) and below you can see those with frequencies more than 50.

```

490
('Create Purchase Order Item', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Clear Invoice') 2320
('Create Purchase Order Item', 'Change Price', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Clear Invoice') 172
('Create Purchase Order Item', 'Change Quantity', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Clear Invoice') 81
('Create Purchase Order Item', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Clear Invoice') 1388
('Create Purchase Order Item', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Remove Payment Block', 'Clear Invoice') 528
('Create Purchase Order Item', 'Record Goods Receipt', 'Record Service Entry Sheet', 'Vendor creates invoice', 'Record Invoice Receipt', 'Clear Invoice') 283
('Create Purchase Order Item', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Remove Payment Block', 'Clear Invoice') 291
('Create Purchase Requisition Item', 'Create Purchase Order Item', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Clear Invoice') 246
('Create Purchase Order Item', 'Change Price', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Clear Invoice') 77
('Create Purchase Order Item', 'Change Price', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Remove Payment Block', 'Clear Invoice') 102
('Create Purchase Order Item', 'Change Quantity', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Clear Invoice') 131
('Create Purchase Order Item', 'Record Goods Receipt', 'Record Service Entry Sheet', 'Vendor creates invoice', 'Record Invoice Receipt', 'Remove Payment Block', 'Clear Invoice') 141
('Create Purchase Requisition Item', 'Create Purchase Order Item', 'Record Goods Receipt', 'Vendor creates invoice', 'Record Invoice Receipt', 'Clear Invoice') 139
('Create Purchase Requisition Item', 'Create Purchase Order Item', 'Vendor creates invoice', 'Record Goods Receipt', 'Record Invoice Receipt', 'Remove Payment Block', 'Clear Invoice')

```

But the results as you can see were very scattered.

After that I figured out that there are some activities starting with a SRM prefix, So I decided to make clusters based on having or not having SRM in each of 4 categories.

And after analyzing each column to see how many values each can have, the column

case:Item Type could have 6 different values of :

1. service
2. Standard
3. Subcontracting
4. Third_party
5. consignment
6. limit

The other column values were too much to use as a clustering metric.

Clusters

	Events
1.1 3way_before_SRM_Standard	8230
1.2.1 3way_before_not_SRM_Standard	905443
1.2.2 3way_before_not_SRM_Third_party	21729
1.2.3 3way_before_not_SRM_Subcontracting	10405
2.1.1 3way_after_SRM_standard	1163
2.1.2 3way_after_SRM_Service	4033
2.2.1 3way_after_not_SRM_standard	35011
2.2.2 3way_after_not_SRM_Third_party&Subcontracting	1345
2.2.3 3way_after_not_SRM_Service	7897
3 filtered2_log_2way	1016
4 filtered_log_Consignment	32591

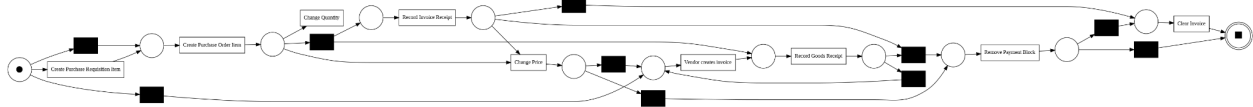
Process Models(BPMNs)

Process Discovery algorithms want to find a suitable process model that describes the order of events/activities that are executed during a process execution.

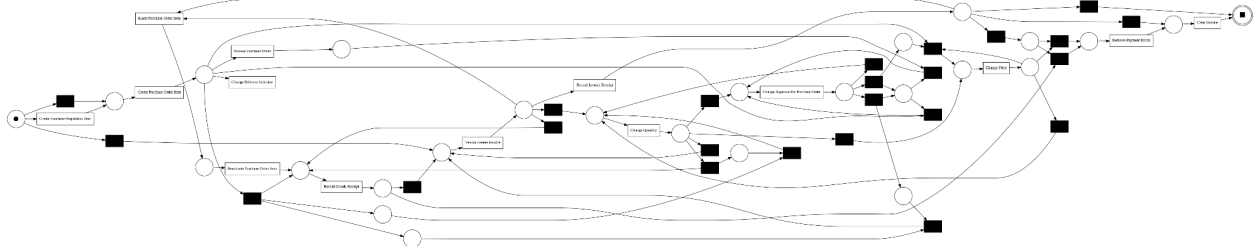
There are 3 types of mining algorithms:

1. Alpha miner

not_SRM_Third_party_variant_log_3way_invoice_before_good_receipt



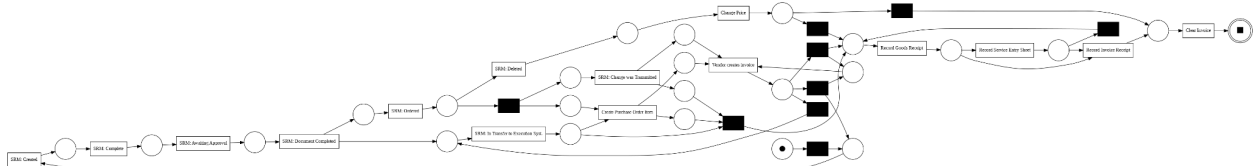
not_SRM_Subcontracting_variant_log_3way_invoice_before_good_receipt



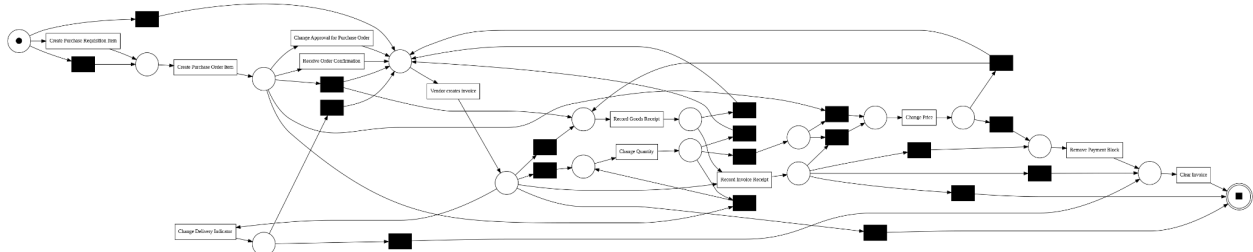
SRM_Standard_variant_log_3way_invoice_after_good_receipt



SRM_Service_variant_log_3way_invoice_after_good_receipt



not_SRM_Standard_variant_log_3way_invoice_after_good_receipt



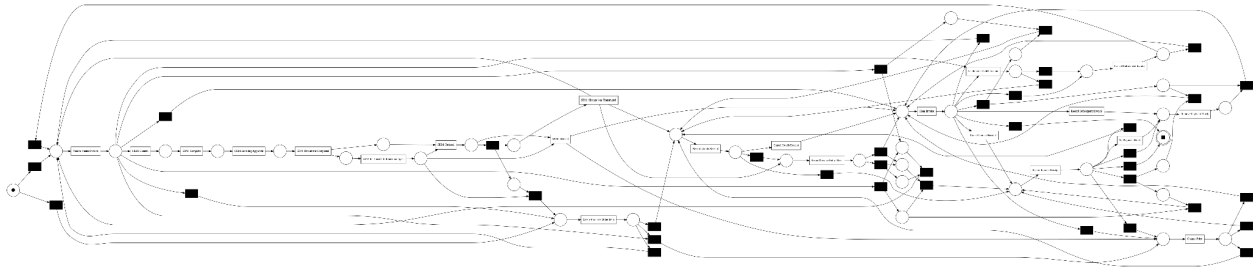
Cluster 2.2.2

not_SRM_Third_party_variant_log_3way_invoice_after_good_receipt



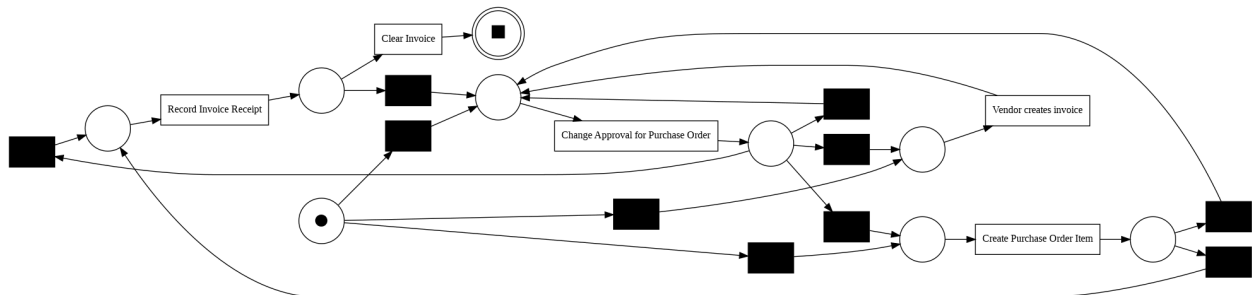
Cluster 2.2.3

not_SRM_Service_variant_log_3way_invoice_after_good_receipt



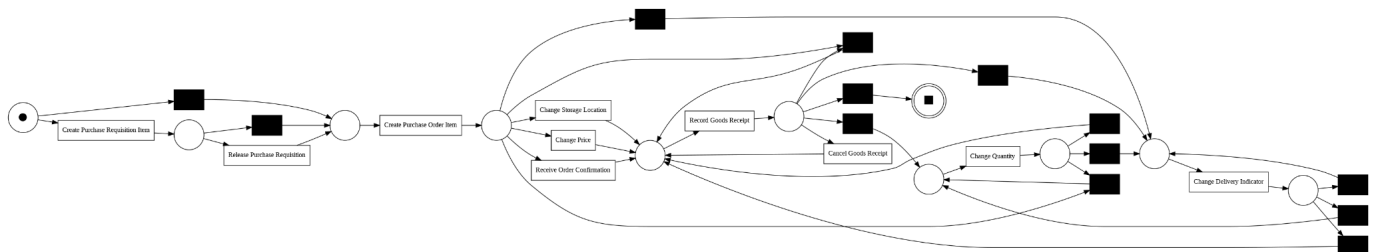
Cluster 3

2way



Cluster 4

Consignment



Validation of case group and process model quality

Here I check the satisfaction in fitness, precision and simplicity.

	precision	fitness	simplicity
cluster			
1.1	0.940000	0.960000	0.790000
1.2.1	0.830000	0.940000	0.500000
1.2.2	0.980000	0.890000	0.660000
1.2.3	0.990000	0.820000	0.560000
2.1.1	0.940000	0.940000	0.760000
2.1.2	0.960000	0.850000	0.650000
2.2.1	0.990000	0.970000	0.540000
2.2.2	0.990000	0.990000	0.730000
2.2.3	0.900000	0.850000	0.530000
3	0.860000	0.920000	0.660000
4	0.990000	0.830000	0.540000
mean	0.942727	0.905455	0.629091

Phase 2 - Answering challenge 2

Throughput Time Analysis

- Using time_related indicators for measuring the performance of an organization
- Average time that a PO item spends in the process = throughput time
- Here we focus on answering "what is the throughput of the invoicing process?" Which is the time between GR, IR, clear invoice(payment)
In 2way : the time between IR to CI
In 3way before : the time between IR and CI
In 3way after: the time between GR and CI
In consignment there is no invoicing procedure
- I used the **filter_between** function in pm4py to filter out those parts which are needed for invoicing procedures, and then the **get_all_case_durations** function to get the throughput time for each case. Finally getting the average among all throughput times for each category(in days)

Category	AVG Throughput time(days) Of invoicing
3way_before	59
3way_after	49
2way	10

Phase 3 - Answering challenge 3

Deviations

I think We can consider the incomplete and incompliance cases as deviations here(Those we filtered first, in order to have a better model).So according to preprocessing phase we can come up to following table as our deviations:

Rule \ Category	2way	3way_after	3way_before	Consignment	Severity
#GR > 0	-	1969	35589	2074	0.04

#CI > 0 #IR > 0	0	161353	159342	-	0.35
PO item value = Cumulated value of IR / #IR	-	1016	0	-	Almost 0
Cumulated value of GRs = Cumulated value of IR	-	5410	4578	-	0.01
#GR = #IR	-	0	65303	-	0.07
End activity : CI	9863	143124	207460	-	0.39
End activity : Record GR	-	-	-	1418	Almost 0
Not having fewer IR than GR and vice versa	-	79860	39612	-	0.13
Total	9863	392732	511884	3492	917971

They didn't mention how to calculate the severity of deviations, so i consider for each deviation:
 $\frac{\text{\#events not following the rule}}{\text{total \#events not following all the rules}}$

Conclusion

All the 3 challenges mentioned are successfully answered.

First I tried to filter the log based on rules mentioned by the company, Then I tried to check the variants among the log and make some clusters among them to have a more precise business model for each of them and then I made the business model (Heuristic miner) for each of the clusters.

In the second phase I tried to calculate the average throughput time of invoicing procedures in each category.

And Finally, I wrote the deviations based on the first phase and their deviations.